Universidad Nacional Autónoma de México

Facultad de Ingeniería

Asignatura: Bases de Datos (1644)

Profesor: Ing. Fernando Arreola Franco

Semestre 2026-2

"Tarea 8"

Grupo: 1

Alumna: Bautista Reyes Sofía

No. de cuenta: 321103891

Fecha de entrega: 13 de abril de 2026

Tarea 8

Investigar 6 tipos de datos en postgresql:

Numéricos

Los tipos de datos numéricos permiten almacenar valores numéricos enteros, decimales o de punto flotante, incluyendo números exactos y aproximados.

Capacidad de Almacenamiento:

Depende del tipo:

- smallint: 2 bytes.
- integer: 4 bytes.
- bigint: 8 bytes.
- real: 4 bytes.
- double precision: 8 bytes.
- numeric/decimal: tamaño variable según la precisión (hasta 131072 dígitos antes del punto decimal y 16383 después).

Formato de representación:

- Enteros: números sin parte decimal.
- Decimales (numeric/decimal): permiten definir precisión y escala → NUMERIC(precision, scale).
- Punto flotante (real, double precision): representación aproximada según estándar IEEE 754.
- También pueden incluir valores especiales como 'Infinity', '-Infinity' y 'NaN'.

Ejemplos:

- smallint → 100: Número entero pequeño dentro del rango permitido (-32768 a 32767).
- integer → 100000: Número entero común, usado en la mayoría de los casos.
- bigint → 9000000000: Número entero muy grande, cuando integer no es suficiente.
- numeric(5,2) → 123.45: Número exacto con 5 dígitos en total y 2 decimales (precisión = 5, escala = 2).
- real → 3.14: Número decimal aproximado con menor precisión.
- double precision → 3.14159265358979: Número decimal aproximado con mayor precisión que real [1], [2].

Carácter

Los tipos de datos de caracteres permiten almacenar cadenas de texto. Incluyen tipos de longitud fija y variable, dependiendo de cómo se definan.

Capacidad de Almacenamiento:

- character(n) / char(n): longitud fija de hasta n caracteresl.
- varchar(n) / character varying(n): longitud variable hasta n caracteres.
- text: longitud ilimitada (hasta aproximadamente 1 GB).

El tamaño real depende de la longitud de la cadena (1 byte extra para cadenas cortas y 4 bytes para largas).

Formato de representación:

- char(n): almacena texto de longitud fija y rellena con espacios si es más corto.
- varchar(n): almacena texto variable sin rellenar espacios.
- text: almacena texto sin límite definido
- Las cadenas no pueden exceder el tamaño definido, excepto si los caracteres extra son espacios (pueden truncarse).

Ejemplos:

- char(4) → 'ok': Se almacena como 'ok ' (rellenado con espacios hasta longitud 4).
- varchar(5) → 'ok': Se almacena exactamente como 'ok', sin espacios adicionales.
- varchar(5) → 'good': Se almacena completo porque no excede el límite.
- varchar(5) → 'too long': Genera error porque excede la longitud máxima permitida.
- text → 'Hola mundo': Puede almacenar texto sin límite específico de longitud [3].

Fecha

Los tipos de datos de fecha y tiempo permiten almacenar fechas, horas, marcas de tiempo y duraciones, siguiendo el calendario gregoriano y soportando diferentes formatos y zonas horarias.

Capacidad de Almacenamiento:

Depende del tipo:

- date: 4 bytes.
- time: 8 bytes.
- timestamp: 8 bytes.

- time with time zone: 12 bytes.
- interval: 16 bytes.

Formato de representación:

- date: solo fecha (año-mes-día).
- time: solo hora (hh:mm:ss).
- timestamp: fecha y hora.
- timestamp with time zone: incluye zona horaria.
- interval: duración de tiempo (años, días, horas, etc.).

Acepta múltiples formatos como ISO 8601 y valores especiales como 'now', 'today', 'yesterday', 'tomorrow'.

Ejemplos:

- date → '1999-01-08': Representa una fecha específica (año, mes y día).
- time → '04:05:06': Representa una hora del día.
- timestamp → '1999-01-08 04:05:06': Combina fecha y hora en un solo valor.
- timestamp with time zone → '1999-01-08 04:05:06-08:00': Incluye la zona horaria (diferencia respecto a UTC).
- interval → '1 year 2 months': Representa una duración de tiempo.
- date → 'today': Valor especial que representa la fecha actual [4].

Boolean (booleano)

El tipo de dato booleano permite almacenar valores lógicos de verdadero o falso.

Capacidad de Almacenamiento:

Ocupa 1 byte.

Formato de representación:

Acepta valores como true y false, así como equivalentes: yes, on, 1 (verdadero) y no, off, 0 (falso) [5].

Ejemplos:

TRUE → representa el valor lógico verdadero.

FALSE → representa el valor lógico falso.

NULL → representa un valor desconocido o indefinido.

También se pueden usar equivalentes como:

```
'yes'::boolean
'no'::boolean
```

En PostgreSQL, 'yes' y 'no' son cadenas de texto, pero al usar ::boolean se convierten explícitamente al tipo booleano, el operador :: se llama conversión de tipo (casting) y se usa para indicar a PostgreSQL que transforme un valor a otro tipo de dato. Tambien PostgreSQL acepta diferentes representaciones de entrada como 'true', 'yes', 'on', '1' y sus equivalentes para falso, pero siempre muestra la salida como 't' o 'f' [6].

JSON / JSONB

Los tipos de datos JSON permiten almacenar información en formato JSON (JavaScript Object Notation), asegurando que los datos cumplan con la estructura válida de este formato.

Capacidad de Almacenamiento:

No tiene un tamaño fijo, depende del contenido almacenado:

- json: guarda el texto original completo.
- jsonb: guarda los datos en formato binario optimizado.

Formato de representación:

- json: conserva exactamente el texto ingresado (incluyendo espacios y orden de claves).
- jsonb: no conserva espacios ni orden, elimina duplicados y optimiza para consultas.

Los datos deben seguir la estructura JSON: objetos , arreglos [], valores como números, texto, booleanos y null.

Ejemplos:

`'5'::json` → Valor JSON simple (número).

`'[1, 2, "foo", null]'::json` → Arreglo con distintos tipos de datos.

`'{"bar": "baz", "balance": 7.77, "active": false}'::json` → Objeto JSON con pares clave-valor.

`'{"foo": [true, "bar"], "tags": {"a": 1, "b": null}}'::json` → Estructura JSON anidada (objetos dentro de arreglos) [7].

Array

Los arrays permiten almacenar múltiples valores de un mismo tipo en una sola columna, pudiendo ser de una o varias dimensiones.

Capacidad de Almacenamiento:

No tienen un tamaño fijo, pueden ser de longitud variable y multidimensionales, y el tamaño declarado no es estrictamente limitado en PostgreSQL, ya que se usa principalmente como documentación.

Formato de representación:

Se representan con llaves y valores separados por comas, y también pueden declararse usando [ ] en el tipo de dato. Pueden ser multidimensionales (arrays dentro de arrays) y se pueden usar valores NULL dentro del arreglo.

Ejemplos:

`{10000, 10000, 10000, 10000}` → Arreglo de enteros (una dimensión).

`{{1,2,3},{4,5,6},{7,8,9}}` → Arreglo bidimensional (matriz).

`ARRAY[10000, 10000, 10000, 10000]` → Forma alternativa de declarar un arreglo.

`ARRAY[['meeting','lunch'],['training','presentation']]` → Arreglo de texto en dos dimensiones [8].

Referencias

[1] PostgreSQL Global Development Group. (2026) 8.1. Numeric Types. PostgreSQL. Consultado en marzo de 2026. [Online]. Available: https://www.postgresql.org/docs/current/datatype-numeric.html

[2] J. Segovia. (2024, Jul.) PostgreSQL data types, tipos de datos más utilizados. Todo PostgreSQL. Consultado en marzo de 2026. [Online]. Available: https://www.todopostgresql.com/postgresql-data-types-los-tipos-de-datos-mas-utilizados/

[3] PostgreSQL Global Development Group. (2026) 8.3. Character Types. PostgreSQL. Consultado en marzo de 2026. [Online]. Available: https://www.postgresql.org/docs/current/datatype-character.html

[4] PostgreSQL Global Development Group 1. (2026) 8.5. Date/Time Types. PostgreSQL. Consultado en marzo de 2026. [Online]. Available: https://www.postgresql.org/docs/current/datatype-datetime.html

[5] C. Custer. (2024, Jan.) PostgreSQL data types: what are they, and when to use each. Cockroach Labs. Consultado en marzo de 2026. [Online]. Available: https://www.cockroachlabs.com/blog/postgres-data-types/

[6] PostgreSQL Global Development Group 2. (2026) 8.6. Boolean Type. PostgreSQL. Consultado en marzo de 2026. [Online]. Available: https://www.postgresql.org/docs/current/datatype-boolean.html

[7] PostgreSQL Global Development Group 3. (2026) 8.14. JSON Types. PostgreSQL. Consultado en marzo de 2026. [Online]. Available: https://www.postgresql.org/docs/current/datatype-json.html

[8] PostgreSQL Global Development Group 4. (2026) 8.15. Arrays. PostgreSQL. Consultado en marzo de 2026. [Online]. Available: https://www.postgresql.org/docs/current/arrays.html